

HW2 Problem 4(b)–4© 影片逐字稿（詳細版）

目標長度：3:30–4:30 範圍：HW2 P4(b) 用 DDPG 解 HalfCheetah-v5；P4© 加上 Clipped Double Q 建議錄製方式：左半畫面 VS Code（顯示 `ddpg_cheetah.py` / `ddpg_cdq_cheetah.py`），右半畫面 W&B / 報告 PDF；切換時口頭明確說「切到 ...」。為錄製方便，每段標明 **【螢幕】** 要呈現的畫面與行號，**【口白】** 為逐字稿。

段落 0 — 開場（約 15 秒）

【螢幕】 投影片首頁：標題「HW2 P4(b)/©：DDPG and DDPG+CDQ on HalfCheetah-v5」、姓名、學號、日期。

【口白】

大家好，我是 [姓名]、學號 [學號]。這支影片會說明 HW2 第四題 (b) 和 ©：先用 vanilla DDPG 解 HalfCheetah-v5，再加上 Clipped Double Q 看訓練上的差別。我會直接搭配程式碼跟 W&B 結果一起講。

段落 1 — P4(b) 網路架構（約 35 秒）

【螢幕】 VS Code 開啟 `ddpg_cheetah.py`，捲到 **L98–L114**（`Actor` 類別），高亮這段。

```
# ddpg_cheetah.py L98–L114 python
class Actor(nn.Module):
    def __init__(self, obs_dim, act_dim, max_action, hidden_size):
        ...
        self.net = nn.Sequential(
            nn.Linear(obs_dim, hidden_size), nn.ReLU(),
            nn.Linear(hidden_size, hidden_size), nn.ReLU(),
            nn.Linear(hidden_size, act_dim),
            nn.Tanh(),
        )
        ...
    def forward(self, state):
        return self.net(state) * self.max_action
```

【口白】

先看 actor。HalfCheetah 的觀測是 17 維、動作是 6 維連續控制。我用兩層 256 ReLU，最後一層接 Tanh 把輸出壓在 -1 到 1 之間，再乘上 `max_action` 把它放大到環境真正的動作範圍。最後一層的權重我用 $\pm 3e-3$ 的小區間初始化，避免一開始輸出就飽和在 ± 1 。

【螢幕】 繼續往下捲到 **L117–L131**（`Critic` 類別），高亮。

【口白】

Critic 把 state 跟 action concat 後丟進兩層 256 ReLU，輸出單一 Q 值。最後一層也是 $\pm 3e-3$ 小範圍初始化。Actor 和 critic 都沒做 layer norm，純粹靠 target network 與 soft update 來穩定訓練。

段落 2 — P4(b) 訓練流程（約 50 秒）

【螢幕】 切到 `ddpg_cheetah.py` **L162–L182**（`DDPGAgent.update`），高亮。

```
# ddpg_cheetah.py L162–L182 python
def update(self, replay_buffer, batch_size):
    state, action, reward, next_state, not_done = replay_buffer.sample(batch_size)
    with torch.no_grad():
```

```
next_action = self.actor_target(next_state)
target_q = self.critic_target(next_state, next_action)
target_q = reward + self.gamma * not_done * target_q

current_q = self.critic(state, action)
critic_loss = F.mse_loss(current_q, target_q)
...
actor_loss = -self.critic(state, self.actor(state)).mean()
...
soft_update(self.actor_target, self.actor, self.tau)
soft_update(self.critic_target, self.critic, self.tau)
```

【口白】

這是核心 update。第一段在 `torch.no_grad()` 裡用 `actor_target` 算 next action，再用 `critic_target` 算 Q 值，組成 TD target $y = r + \gamma \cdot (1-d) \cdot \bar{Q}(s', \bar{\pi}(s'))$ 。Critic loss 是 current Q 和這個 target 的 MSE。

Actor loss 用 deterministic policy gradient：把 actor 的輸出餵進 critic 取負平均。注意這裡是線上的 critic，不是 target critic。最後對 actor 跟 critic 兩個 target 各做一次 Polyak soft update， τ 設 0.005。

【螢幕】切到 `ddpg_cheetah.py` **L232–L254**（main loop 中 `for step in progress` 那段），高亮 `start_steps` 隨機探索與 `agent.update` 呼叫。

【口白】

訓練主迴圈這裡有兩個關鍵：前 1 萬步用 `env.action_space.sample()` 純隨機填 replay buffer，避免一開始就 overfit 到一個壞策略。1 萬步之後改用 actor 加 Gaussian noise，noise std 是 action bound 的 0.1。每跑一個環境步就更新一次 actor 與 critic。

【螢幕】捲到 **L289–L311**（`parse_args`），停在 hyperparameter 一覽。

【口白】

超參數： γ 0.99、 τ 0.005、actor lr 與 critic lr 都是 $3e-4$ 、batch size 256、replay 1M、hidden size 256、隨機 seed 42、總共 50 萬步、每 1 萬步用 20 條 episode 評估一次。

段落 3 — P4(b) 結果（約 35 秒）

【螢幕】切到瀏覽器 W&B：<https://wandb.ai/.../ddpg-halfcheetah/runs/rxkfkqsc>，先看 overview，再切到左側選 `eval/mean_return`。

【口白】

這是 vanilla DDPG 的 W&B run。先看 `eval/mean_return`：大概在 15 萬步左右就跨過題目要求的 5,000 門檻，之後一路爬到第 50 萬步。最終 20-episode 評估平均是 8536.79，落在助教提示的 6,000–10,000 區間。

中段大約 13 萬步附近有一段明顯的 drawdown，這是 vanilla DDPG 常見現象——單一 critic 容易把 Q 值高估，target 被 overshoot 推過頭，actor 也跟著被誤導，後面才慢慢回穩。這個 drawdown 等等會跟 CDQ 對比就更明顯。

【螢幕】切到 `answer.pdf` 第 4(b) 結果表，停 2 秒。

【口白】

報告裡也列了 best checkpoint 的路徑，actor 和 critic 兩個 `.pth` 都存在 `preTrained_online/`。

段落 4 — P4© CDQ 設計差異（約 50 秒）

【螢幕】切到 `ddpg_cdq_cheetah.py` **L149–L160**（CDQ agent 的 critic 初始化），高亮。

```
# ddpq_cdq_cheetah.py L149–L160
self.critic1 = Critic(obs_dim, act_dim, args.hidden_size).to(device)
```

python

```
self.critic2 = Critic(obs_dim, act_dim, args.hidden_size).to(device)
self.critic1_target = Critic(...).to(device)
self.critic2_target = Critic(...).to(device)
self.critic_optim = Adam(
    list(self.critic1.parameters()) + list(self.critic2.parameters()),
    lr=args.critic_lr,
)
```

【口白】

CDQ 跟 vanilla DDPG 的第一個差別：維護兩個獨立初始化的 critic，加上各自的 target critic。兩個 critic 共用一個 Adam optimizer 一起更新。

【螢幕】 捲到 **L172–L195**（`CDQAgent.update` 的 target 計算 + critic loss），高亮。

```
# ddpq_cdq_cheetah.py L172–L195
with torch.no_grad():
    noise = torch.randn_like(action) * self.policy_noise * max_action
    noise = torch.clamp(noise, -self.noise_clip * max_action,
                        self.noise_clip * max_action)

    next_action = self.actor_target(next_state) + noise
    next_action = torch.max(torch.min(next_action, max_action), -max_action)

    target_q1 = self.critic1_target(next_state, next_action)
    target_q2 = self.critic2_target(next_state, next_action)
    target_q = torch.min(target_q1, target_q2)
    target_q = reward + self.gamma * not_done * target_q

current_q1 = self.critic1(state, action)
current_q2 = self.critic2(state, action)
critic1_loss = F.mse_loss(current_q1, target_q)
critic2_loss = F.mse_loss(current_q2, target_q)
```

【口白】

這段是 CDQ 的核心。先在 target action 上加一個 clipped Gaussian noise，policy noise std 設 0.2、clip 範圍 ±0.5、再 clip 回 action bound。這就是 target policy smoothing，讓 critic 對 target 附近的動作做平滑，避免對特定動作過擬合。

然後分別用兩個 target critic 算 `target_q1`、`target_q2`，取最小值當成 bootstrap target。這個 min 是 CDQ 壓制 Q 高估的關鍵：如果其中一個 critic 暫時樂觀，min 會把它砍掉。最後兩個線上 critic 都用同一個 target 算 MSE loss。

【螢幕】 捲到 **L197–L207**（actor delayed update），高亮。

【口白】

第三個差別是 actor delayed update，每兩次 critic 更新才更新一次 actor 跟所有 target，用的還是 `critic1` 算 policy gradient。除了 policy noise、noise clip、policy freq 這三個參數之外，其他超參數和 (b) 完全一樣，比較才公平。

段落 5 — P4(b) vs P4© 比較與觀察（約 50 秒）

【螢幕】 開啟 `figures/p4_eval_compare.png`（兩條 eval curve 疊在一起），全畫面顯示。

【口白】

把兩個 run 的 evaluation curve 疊在一起。藍色是 vanilla DDPG，橘色是 CDQ。第一個觀察：CDQ 大約在 11 萬步就跨過 5,000 門檻，比 vanilla 早了大約 4 萬步。第二個觀察：vanilla 在 13 萬步附近那段大幅 drawdown，在 CDQ 完全沒有出現，CDQ 只有在 12 萬左右一個小震盪、35 萬左右一個短暫凹陷，整體穩定很多。這完全符合 clipped double Q 的理論預期：取兩個 target 的 min 把 bootstrap target 壓低，bias 小、actor 更新方向不會被誇大的 Q 值帶歪。

【螢幕】 切到 `figures/p4_critic_loss_compare.png`（critic Bellman loss，log scale）。

【口白】

再看 critic loss 圖 (log scale)。Vanilla 的 Bellman loss 收斂到比 CDQ 兩個 critic 都略低。但這不代表 vanilla 學得比較好——它只是 fit 在一個被高估的 target 上，loss 小但 target 本身是錯的。CDQ 的 critic1、critic2 loss 走勢幾乎重疊，可是它們在原始 Q 值上其實會分歧，這就是 ensemble 的價值：取 min 之後 bias 被壓住。

不過要誠實說，在這個 seed 下，vanilla 最終分數 8536，反而比 CDQ 的 8223 高一點點。差異主要表現在「訓練穩定度」而不是「峰值」——換 seed 或拉長訓練步數，CDQ 的優勢通常會更明顯。

段落 6 — 總結 (約 20 秒)

【螢幕】 投影片，三點 bullet：

- Vanilla DDPG 已能解 HalfCheetah，但訓練曲線會出現 Q 高估造成的 drawdown
- CDQ 跨門檻更快、訓練更穩定 (drawdown 消失)
- 最終分數兩者接近，差異主要在穩定性，不是峰值

【口白】

總結三點：第一，vanilla DDPG 已經能解 HalfCheetah，但中段會有明顯 drawdown；第二，加上 CDQ 之後跨 5,000 門檻速度更快，訓練曲線更穩定；第三，最終分數兩者接近，CDQ 的優勢主要表現在穩定度而不是峰值。所有超參數、checkpoint 路徑、W&B 連結都列在報告 PDF 裡，謝謝大家。

錄製 / 後製檢查表

- ☐ 螢幕錄影 ≥ 1080p，VS Code 字級調到看得清楚 (建議 14–16pt)
- ☐ VS Code 開啟 minimap 關閉、行號開啟，方便對照逐字稿提到的行號
- ☐ W&B 切到正確 run：vanilla = `rxkfkqsc`、CDQ = `lvdt216v`
- ☐ 兩條 eval curve 疊圖 (`figures/p4_eval_compare.png`) 與 critic loss 圖 (`figures/p4_critic_loss_compare.png`) 預先打開在 Preview，避免錄影時找檔案
- ☐ 麥克風測試：開頭講第一句後回放確認音量
- ☐ 影片總長控制在 3:30–4:30
- ☐ 輸出檔名建議：`hw2_p4_video_[學號].mp4`
- ☐ 上傳前再播放一次，確認沒有麥克風雜音、沒有露出個人帳號 token